AI Application Lecture 16

3D Rendering Objective

SUZUKI, Atsushi[a]

[a] I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

From image generation to 3D generation

In image generation, the goal is to generate a 3 or 4-channel image. We can change the resolution by changing the noise generation with help of variable-shape architecture, i.e., attention, convolutional layer, RoPE.

What

From image generation to 3D generation

In image generation, the goal is to generate a 3 or 4-channel image. We can change the resolution by changing the noise generation with help of variable-shape architecture, i.e., attention, convolutional layer, RoPE.

What is the goal of 3D rendering, and how can we make the resolution adaptable to applications?

Learning outcomes for this lecture

At the end of this lecture, you will be able to

Learning outcomes for this lecture

At the end of this lecture, you will be able to

- **Formulate** the goal of 3D rendering.

Learning outcomes for this lecture

At the end of this lecture, you will be able to

- **Formulate** the goal of 3D rendering.
- **Construct** a scheme to adapt resolutions to application situations in 3D rendering. **Explain** the strategy of 3D rendering AI.

By the end of this lecture, you should be able to:

- Mathematically define a **triangle mesh** and explain how its data structure differs from a fixed-size image tensor.
- Explain how an **implicit 3D field** and a **surface extractor** connect a latent generative model to variable-size mesh generation.
- Describe the complete condition-to-mesh pipeline of a 3D generative AI system, using Hunyuan3D as a concrete example.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} \coloneqq \{1, \ldots, k\}$

Function:

- $f : \mathcal{X} \to \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x.

Definition:

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\boldsymbol{a} = (a_1, a_2, \dots)$.

- A function $\boldsymbol{a} : [1, n]_{\mathbb{Z}} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

Vector: Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- i-th element is v_i.

Matrix: Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- (i, j)-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

Tensor: Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Question: What is the goal in 3D rendering?

Difference from image generation.

In image generation, our objective was to output a 3rd-order tensor, i.e., three matrices corresponding to red channel, green channel, and blue channel.

What

Difference from image generation.

In image generation, our objective was to output a 3rd-order tensor, i.e., three matrices corresponding to red channel, green channel, and blue channel.

What is the objective in 3D rendering?

What should a 3D generative model output?

A common representation of a renderable 3D surface is a **triangle mesh** (e.g., `Trimesh` in Python, GLB file).

What should a 3D generative model output?

A common representation of a renderable 3D surface is a **triangle mesh** (e.g., `Trimesh` in Python, GLB file).

It consists of

What should a 3D generative model output?

A common representation of a renderable 3D surface is a **triangle mesh** (e.g., `Trimesh` in Python, GLB file).

It consists of

- The vertex matrix $\boldsymbol{V} \in \mathbb{R}^{N_V,3}$ storing the coordinates of the N_V vertices.

What should a 3D generative model output?

A common representation of a renderable 3D surface is a **triangle mesh** (e.g., `Trimesh` in Python, GLB file).

It consists of

- The vertex matrix $\boldsymbol{V} \in \mathbb{R}^{N_V,3}$ storing the coordinates of the N_V vertices.
- The face matrix $\boldsymbol{F} \in \{0, 1, ..., N_V - 1\}^{N_F,3}$ stores three vertex indices for each triangular face.

What should a 3D generative model output?

A common representation of a renderable 3D surface is a **triangle mesh** (e.g., `Trimesh` in Python, GLB file).

It consists of

- The vertex matrix $\boldsymbol{V} \in \mathbb{R}^{N_V,3}$ storing the coordinates of the N_V vertices.
- The face matrix $\boldsymbol{F} \in \{0, 1, ..., N_V - 1\}^{N_F,3}$ stores three vertex indices for each triangular face.

Here, the ith row $\begin{bmatrix} f_{i,0} & f_{i,1} & f_{i,2} \end{bmatrix}$ of $\boldsymbol{F}$ indicates the three vertex indices of the ith triangle.

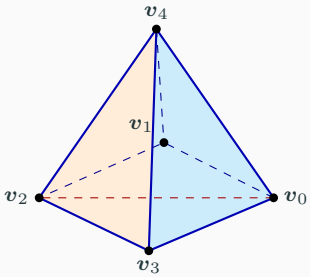
In other words, the three vertices of the ith triangle face is $\boldsymbol{v}_{f_{i,0}}, \boldsymbol{v}_{f_{i,1}}, \boldsymbol{v}_{f_{i,2}}$.

Pyramid-shape example

Example (Five vertices and six triangular faces)

We can construct a pyramid-shape mesh as follows:

$$\boldsymbol{V} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \boldsymbol{F} = \begin{pmatrix} 0 & 2 & 1 \\ 0 & 3 & 2 \\ 0 & 1 & 4 \\ 1 & 2 & 4 \\ 2 & 3 & 4 \\ 3 & 0 & 4 \end{pmatrix}.$$



The first 4 rows of $\boldsymbol{V}$ are vertices of the square base. The first 2 rows of $\boldsymbol{F}$ form the square base, divided into two right isosceles triangles. The other four rows form the lateral faces.

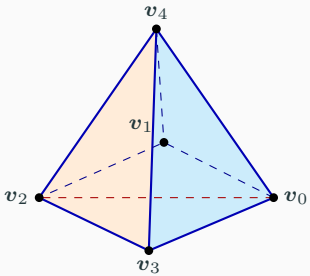
Pyramid-shape example

Exercise (Four triangular lateral faces)

Consider the same square pyramid as in the preceding example, with the same five vertices and vertex matrix $\boldsymbol{V}$.

Remove the two triangles forming the square base, retaining only the four triangular lateral faces.

Write the resulting face matrix $\boldsymbol{F} \in \{0, 1, 2, 3, 4\}^{4,3}$ using zero-based vertex indices.



Pyramid-shape example

Answer:

$$\boldsymbol{F} = \begin{pmatrix} 0 & 1 & 4 \\ 1 & 2 & 4 \\ 2 & 3 & 4 \\ 3 & 0 & 4 \end{pmatrix}.$$

Each row specifies one lateral triangle, and the two base triangles $(0, 2, 1)$ and $(0, 3, 2)$ are excluded. The vertex matrix $\boldsymbol{V}$ is unchanged from the preceding example.

This answer also shows that $\boldsymbol{V}$ does not uniquely determine a triangle mesh.

Mesh generation — Answer

Question: What is the goal in 3D rendering?

Answer:

Mesh generation — Answer

Question: What is the goal in 3D rendering?

Answer: The objective is to generate a set of 3D faces. They are represented as a vertex matrix and face matrix.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

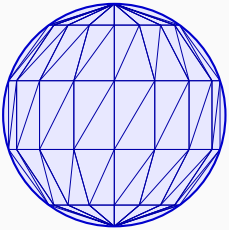
5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

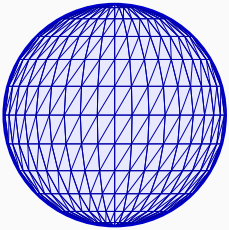
Question: The required resolution of mesh depends on applications and may change at the inference phase. How

Question: The required resolution of mesh depends on applications and may change at the inference phase. How can we make the resolution adaptable?

The resolution may be determined at the inference phase.

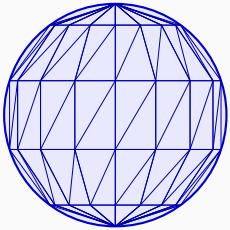


Coarse triangle mesh

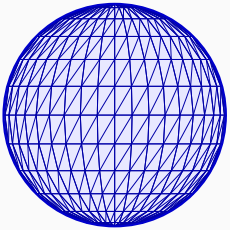


Fine triangle mesh

The resolution may be determined at the inference phase.



Coarse triangle mesh

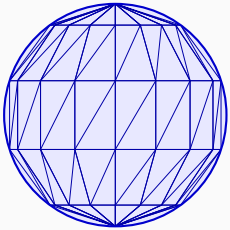


Fine triangle mesh

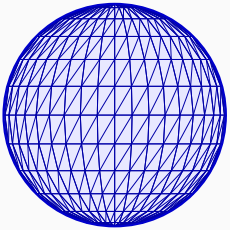
The required resolution depends on applications (e.g., 300 faces in the 2 player mode, 200 faces in the 4 player mode.). We design a pipeline so that one NN can generate 3D mesh at arbitrary resolutions.

How

The resolution may be determined at the inference phase.



Coarse triangle mesh



Fine triangle mesh

The required resolution depends on applications (e.g., 300 faces in the 2 player mode, 200 faces in the 4 player mode.). We design a pipeline so that one NN can generate 3D mesh at arbitrary resolutions.

How can we realize adaptable resolution mesh generation?

Solution: generating a scalar field

In a 3D rendering AI pipeline, a NN generates a scalar **3D field** instead of directly generating mesh.

Solution: generating a scalar field

In a 3D rendering AI pipeline, a NN generates a scalar **3D field** instead of directly generating mesh.

Here, a 3D field is a scalar function $f : \mathbb{R}^3 \to \mathbb{R}$ such that the following hopefully holds:

$$\begin{cases} f(\boldsymbol{q}) \leq \ell & \text{if } \boldsymbol{q} \text{ is inside the object,} \\ f(\boldsymbol{q}) > \ell & \text{if } \boldsymbol{q} \text{ is outside the object.} \end{cases}$$

Here, $\ell \in \mathbb{R}$ is called the **extraction level**.

Outline of mesh generation from a scalar field

Outline of mesh generation from a scalar field

- Generate a scalar 3D field.

Outline of mesh generation from a scalar field

- Generate a scalar 3D field.
- Evaluate the field's value on 3D regular grid points.

Outline of mesh generation from a scalar field

- Generate a scalar 3D field.
- Evaluate the field's value on 3D regular grid points.
- Locate contour points (intersections with the level set) on 3D grid edges.

Outline of mesh generation from a scalar field

- Generate a scalar 3D field.
- Evaluate the field's value on 3D regular grid points.
- Locate contour points (intersections with the level set) on 3D grid edges.
- Construct 3D mesh by connecting the contour points.

Mesh generation — 2D analogy

Since it isn’t easy to imagine 3D from the beginning, let’s consider a 2D case first.

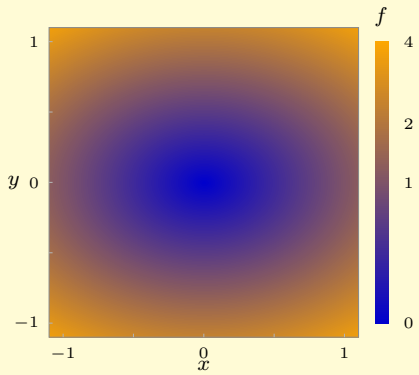
Example: a scalar field in two dimensions

Example (2D Field values in the plane)

Let $f : \mathbb{R}^2 \to \mathbb{R}, \quad f(x, y) = x^2 + 2y^2$.
For example,

$$f(0,0) = 0, \quad f(1,0) = 1, \quad f(0,1) = 2.$$

The diagram shows f on $[-1.1, 1.1]^2$;
the color bar gives the scalar values.



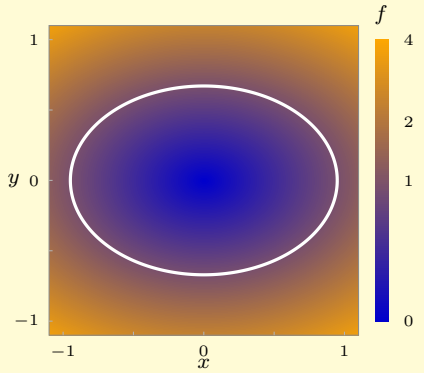
Example: a level set in two dimensions

Example (A contour defined by an equation)

Choose the extraction level $\ell = \frac{9}{10}$. The corresponding level set is

$$\mathcal{S}_\ell = \left\{ (x, y) \in \mathbb{R}^2 \;\middle|\; x^2 + 2y^2 = \frac{9}{10} \right\}.$$

The white curve consists of exactly those points where $f(x, y) = \ell$.



Example: sampling a two-dimensional field

Example (Field values on a regular grid)

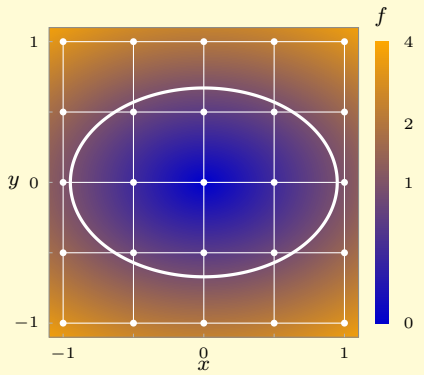
Let $i, j \in [0, 4]_{\mathbb{Z}}$ and set

$$\boldsymbol{q}_{ij} = \left(-1 + \frac{i}{2}, -1 + \frac{j}{2}\right), \qquad G_{ij} = f(\boldsymbol{q}_{ij}).$$

The 5×5 array $\boldsymbol{G} = (G_{ij})$ is

$$\boldsymbol{G} = \begin{pmatrix} 3 & \frac{3}{2} & 1 & \frac{3}{2} & 3 \\ \frac{9}{4} & \frac{3}{4} & \frac{1}{4} & \frac{3}{4} & \frac{9}{4} \\ 2 & \frac{1}{2} & 0 & \frac{1}{2} & 2 \\ \frac{9}{4} & \frac{3}{4} & \frac{1}{4} & \frac{3}{4} & \frac{9}{4} \\ 3 & \frac{3}{2} & 1 & \frac{3}{2} & 3 \end{pmatrix}.$$

The dots mark the 25 grid points; the white curve is the level set $f = \frac{9}{10}$.



Example: locating a contour point on a grid edge

Example (Linear interpolation)

Consider the edge from $\boldsymbol{a} = (\frac{1}{2}, 0)$ to $\boldsymbol{b} = (1, 0)$. Its endpoint values are

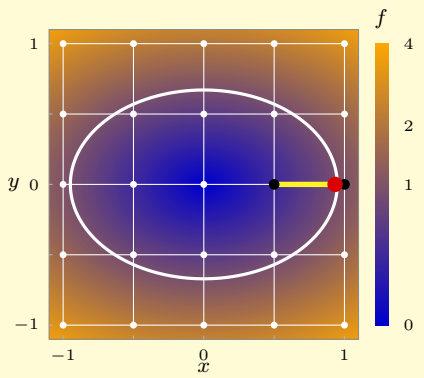
$$f(\boldsymbol{a}) = \frac{1}{4} < \frac{9}{10}, \qquad f(\boldsymbol{b}) = 1 > \frac{9}{10}.$$

Linear interpolation uses

$$t = \frac{\ell - f(\boldsymbol{a})}{f(\boldsymbol{b}) - f(\boldsymbol{a})} = \frac{\frac{9}{10} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{13}{15}.$$

The extracted point is

$$\boldsymbol{p} = (1-t)\boldsymbol{a} + t\boldsymbol{b} = \left(\frac{14}{15}, 0\right).$$

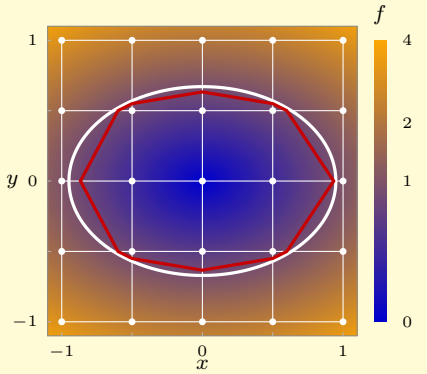


Example: a line-segment mesh in two dimensions

Example (Vertices and connectivity of the extracted contour)

Let $\boldsymbol{V} \in \mathbb{R}^{12\times 2}$ have rows $\boldsymbol{p}_1, \ldots, \boldsymbol{p}_{12}$, where

$$\begin{aligned}
\boldsymbol{p}_1 &= (0, \tfrac{19}{30}), & \boldsymbol{p}_2 &= (\tfrac{1}{2}, \tfrac{11}{20}),\\
\boldsymbol{p}_3 &= (\tfrac{3}{5}, \tfrac{1}{2}), & \boldsymbol{p}_4 &= (\tfrac{14}{15}, 0),\\
\boldsymbol{p}_5 &= (\tfrac{3}{5}, -\tfrac{1}{2}), & \boldsymbol{p}_6 &= (\tfrac{1}{2}, -\tfrac{11}{20}),\\
\boldsymbol{p}_7 &= (0, -\tfrac{19}{30}), & \boldsymbol{p}_8 &= (-\tfrac{1}{2}, -\tfrac{11}{20}),\\
\boldsymbol{p}_9 &= (-\tfrac{3}{5}, -\tfrac{1}{2}), & \boldsymbol{p}_{10} &= (-\tfrac{14}{15}, 0),\\
\boldsymbol{p}_{11} &= (-\tfrac{3}{5}, \tfrac{1}{2}), & \boldsymbol{p}_{12} &= (-\tfrac{1}{2}, \tfrac{11}{20}).
\end{aligned}$$



Let $\boldsymbol{F} \in [1, 12]_{\mathbb{Z}}^{12\times 2}$ have rows $(1,2), (2,3), \ldots, (11,12), (12,1)$. Each row (i, j) represents the line segment

$$\{(1-t)\boldsymbol{p}_i + t\boldsymbol{p}_j \mid 0 \le t \le 1\}.$$

3D field

Similarly, we can generate a 3D mesh from 3D field $f : \mathbb{R}^3 \to \mathbb{R}$.

A surface is represented as a level set

Choose a scalar $\ell \in \mathbb{R}$, called the **extraction level**.

A surface is represented as a level set

Choose a scalar $\ell \in \mathbb{R}$, called the **extraction level**.

The surface at level ℓ is

$$\mathcal{S}_\ell \coloneqq \left\{ \boldsymbol{q} \in \mathbb{R}^3 \;\middle|\; f(\boldsymbol{q}) = \ell \right\}.$$

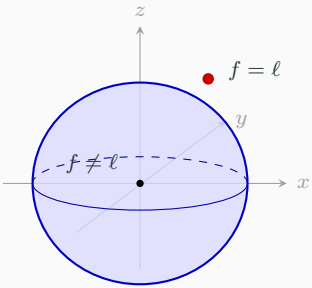
A surface is represented as a level set

Choose a scalar $\ell \in \mathbb{R}$, called the **extraction level**.

The surface at level ℓ is

$$\mathcal{S}_\ell \coloneqq \left\{ \boldsymbol{q} \in \mathbb{R}^3 \;\middle|\; f(\boldsymbol{q}) = \ell \right\}.$$

Thus the surface is specified by an equation rather than by an explicit list of triangles.



Example: a surface extracted from a 3D field

Example (An ellipsoidal level surface)

Consider the scalar field $f(x, y, z) = x^2 + y^2 + z^2$.

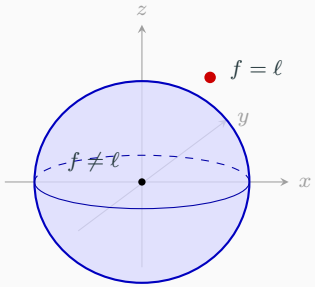
At extraction level $\ell = 1$, the surface is

$$\mathcal{S}_1 = \{(x, y, z) \in \mathbb{R}^3 \mid x^2 + y^2 + z^2 = 1\}.$$

For instance,

$$f(0, 0, 0) = 0, \qquad f(1, 0, 0) = 1, \qquad f(2, 0, 0) = 4.$$

Consequently, $(1, 0, 0)$ belongs to $\mathcal{S}_1$, whereas $(0, 0, 0)$ or $(2, 0, 0)$ does not.



Sampling the field on a finite grid

We cannot evaluate the value of the 3D field at every point in $\mathbb{R}^3$. Instead, we evaluate the values at grid points.

Sampling the field on a finite grid

We cannot evaluate the value of the 3D field at every point in $\mathbb{R}^3$. Instead, we evaluate the values at grid points.

Choose a bounding box with opposite corners $\boldsymbol{b}_{\min}, \boldsymbol{b}_{\max} \in \mathbb{R}^3$, and let $R \in \mathbb{Z}_{>0}$ denote the grid resolution.

Sampling the field on a finite grid

We cannot evaluate the value of the 3D field at every point in $\mathbb{R}^3$. Instead, we evaluate the values at grid points.

Choose a bounding box with opposite corners $\boldsymbol{b}_{\min}, \boldsymbol{b}_{\max} \in \mathbb{R}^3$, and let $R \in \mathbb{Z}_{>0}$ denote the grid resolution.

For $i, j, k \in [0, R]_{\mathbb{Z}}$, define

$$\boldsymbol{q}_{ijk} \coloneqq \boldsymbol{b}_{\min} + \frac{(i, j, k)}{R} \odot (\boldsymbol{b}_{\max} - \boldsymbol{b}_{\min}).$$

Sampling the field on a finite grid

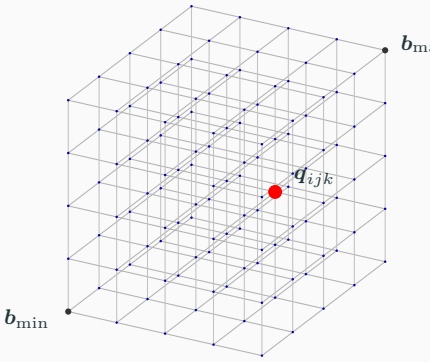
We cannot evaluate the value of the 3D field at every point in $\mathbb{R}^3$. Instead, we evaluate the values at grid points.

Choose a bounding box with opposite corners $\boldsymbol{b}_{\min}, \boldsymbol{b}_{\max} \in \mathbb{R}^3$, and let $R \in \mathbb{Z}_{>0}$ denote the grid resolution.

For $i, j, k \in [0, R]_{\mathbb{Z}}$, define

$$\boldsymbol{q}_{ijk} \coloneqq \boldsymbol{b}_{\min} + \frac{(i, j, k)}{R} \odot (\boldsymbol{b}_{\max} - \boldsymbol{b}_{\min}).$$

Define $G_{ijk} \coloneqq f(\boldsymbol{q}_{ijk})$.



Example: sampling a level surface

Example (Scalar samples across an ellipsoid)

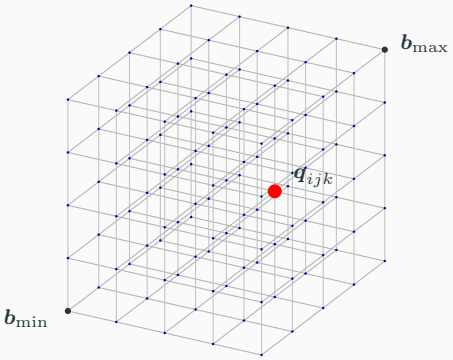
Use $f(x, y, z) = x^2 + 2y^2 + 3z^2$ and $\ell = \frac{4}{5}$.
Choose $\boldsymbol{b}_{\min} = (-1, -1, -1)$, $\boldsymbol{b}_{\max} = (1, 1, 1)$, and $R = 4$. Then

$$\boldsymbol{q}_{ijk} = \left(-1 + \frac{i}{2}, -1 + \frac{j}{2}, -1 + \frac{k}{2}\right).$$

For example,

$$G_{222} = 0 < \ell, \qquad G_{322} = \frac{1}{4} < \ell, \qquad G_{422} = 1 > \ell.$$

On the edge from $\boldsymbol{q}_{322} = (\frac{1}{2}, 0, 0)$ to $\boldsymbol{q}_{422} = (1, 0, 0)$, linear interpolation at level ℓ gives the point $(\frac{13}{15}, 0, 0)$.



Surface extraction

The tensor $\underline{G}$ is still not a triangle mesh.

Surface extraction

The tensor $\underline{G}$ is still not a triangle mesh.
A **surface extractor** converts the sampled level set into vertices and triangular faces.

Surface extraction

The tensor $\underline{\boldsymbol{G}}$ is still not a triangle mesh.
A **surface extractor** converts the sampled level set into vertices and triangular faces.
One standard method is **Marching Cubes** [4].

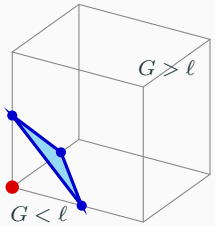
$$(\boldsymbol{V}, \boldsymbol{F}) = \mathrm{MarchingCubes}(\underline{\boldsymbol{G}}, \ell).$$

Surface extraction

The tensor $\underline{\boldsymbol{G}}$ is still not a triangle mesh.
A **surface extractor** converts the sampled level set into vertices and triangular faces.
One standard method is **Marching Cubes** [4].

$$(\boldsymbol{V}, \boldsymbol{F}) = \mathrm{MarchingCubes}(\underline{\boldsymbol{G}}, \ell).$$

If only one vertex on the cube has a different sign, the face is a simple triangle as in the right figure.



Note: the surface extraction can be complicated

If multiple vertices on the cube has a different sign, the situation is quite complicated.

See, e.g.,
`https://www.boristhebrave.com/2018/04/15/marching-cubes-3d-tutorial/`.

Generating mesh from a field

Question: How

Generating mesh from a field

Question: How can we make the resolution of a 3D mesh adaptable?

Answer:

Generating mesh from a field

Question: How can we make the resolution of a 3D mesh adaptable?

Answer: We let a NN to output a **3D field**, instead of a 3D mesh directly. Then,

Generating mesh from a field

Question: How can we make the resolution of a 3D mesh adaptable?

Answer: We let a NN to output a **3D field**, instead of a 3D mesh directly. Then, we can apply a **surface extractor without trainable parameters** to generate a 3D mesh from the 3D field.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

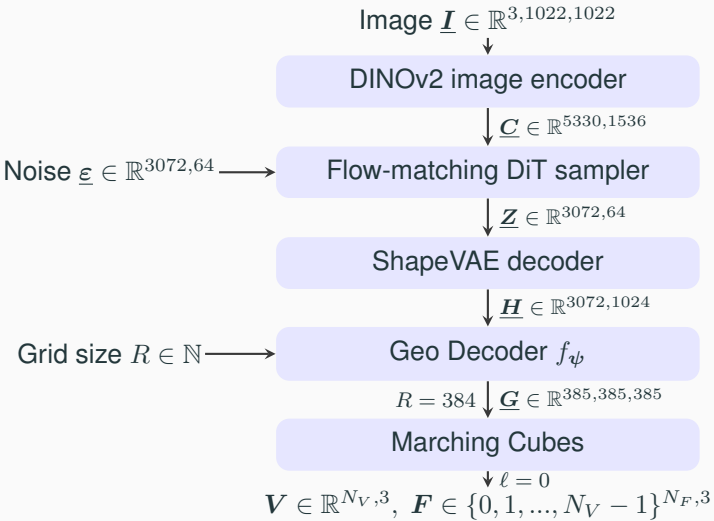
6. Summary

Question: How does a practical 3D rendering AI generate a 3D field and mesh?

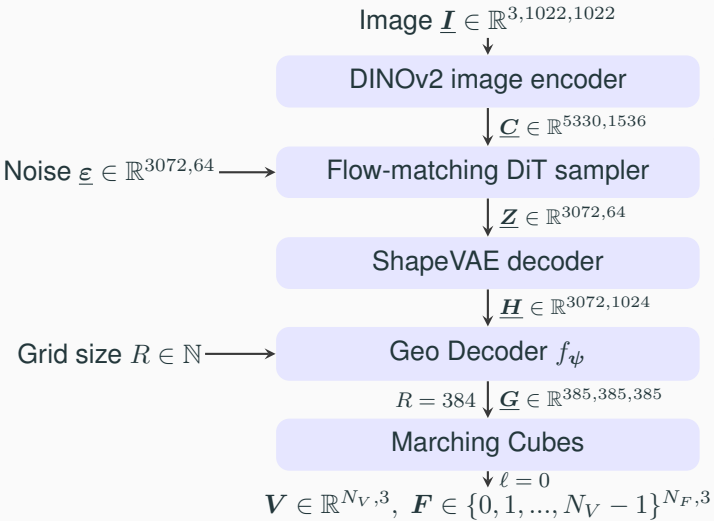
A concrete example: Hunyuan3D

We now use the shape-generation pipeline of Hunyuan3D-2mini Turbo as a concrete example [1, 3].

A concrete example: Hunyuan3D

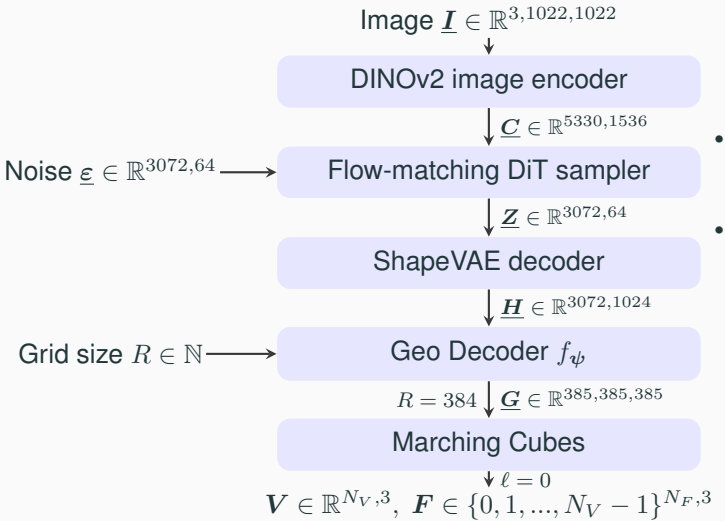


A concrete example: Hunyuan3D



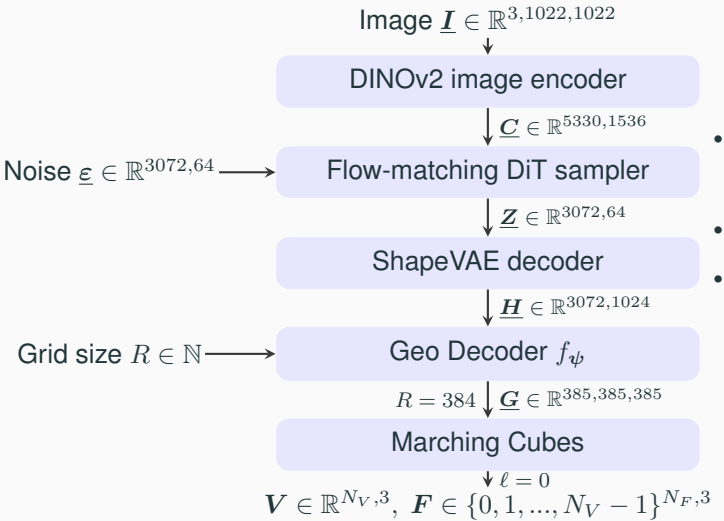
- The image is converted into a tensor $\underline{\boldsymbol{H}}$.

A concrete example: Hunyuan3D



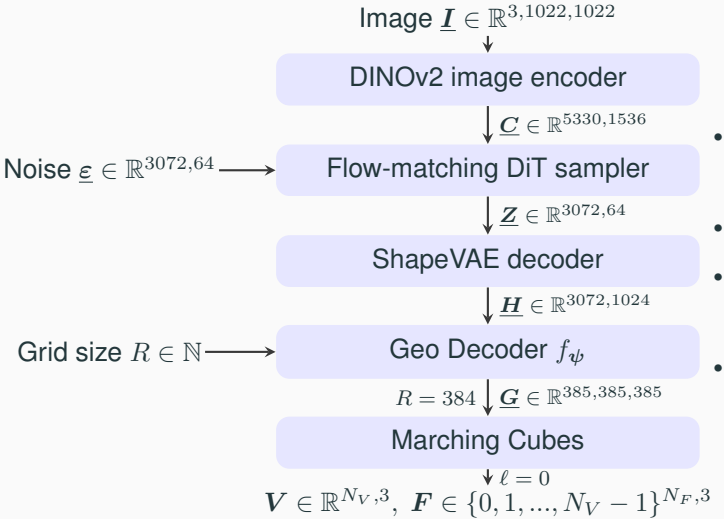
- The image is converted into a tensor $\underline{\boldsymbol{H}}$.
- $f_{\psi}(\cdot, \underline{\boldsymbol{H}})$ defines a field.

A concrete example: Hunyuan3D



- The image is converted into a tensor $\underline{\boldsymbol{H}}$.
- $f_{\psi}(\cdot, \underline{\boldsymbol{H}})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.

A concrete example: Hunyuan3D



- The image is converted into a tensor $\underline{\boldsymbol{H}}$.
- $f_{\psi}(\cdot, \underline{\boldsymbol{H}})$ defines a field.
- Values $\underline{\boldsymbol{G}}$ of the field are evaluated at the grid points.
- mesh are generated from the field value tensor $\underline{\boldsymbol{G}}$.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. What Is the Goal? – Triangle Mesh

4. Adapter from a Latent Diffusion Model – Implicit 3D Field and Surface Extractor

5. The Complete 3D Rendering AI Pipeline – Hunyuan3D as an Example

6. Summary

Summary

Let's return to the three learning outcomes.

- A **triangle mesh** consists of a real-valued vertex matrix $\boldsymbol{V}$ and an integer-valued face matrix $\boldsymbol{F}$. Its numbers of vertices and faces are not fixed in advance.

Summary

Let's return to the three learning outcomes.

- A **triangle mesh** consists of a real-valued vertex matrix $\boldsymbol{V}$ and an integer-valued face matrix $\boldsymbol{F}$. Its numbers of vertices and faces are not fixed in advance.
- An **implicit 3D field** assigns a scalar to each point in 3D space. Sampling a level set of this field and applying a **surface extractor** connects a fixed-size continuous representation to a variable-size mesh.

Summary

Let's return to the three learning outcomes.

- A **triangle mesh** consists of a real-valued vertex matrix $\boldsymbol{V}$ and an integer-valued face matrix $\boldsymbol{F}$. Its numbers of vertices and faces are not fixed in advance.
- An **implicit 3D field** assigns a scalar to each point in 3D space. Sampling a level set of this field and applying a **surface extractor** connects a fixed-size continuous representation to a variable-size mesh.
- Hunyuan3D implements the pipeline

  image $\rightarrow$ image features $\rightarrow$ shape latent $\rightarrow$ geometry memory $\rightarrow$ implicit field $\rightarrow$ tria

References i

[1] Tencent Hunyuan Team.
Hunyuan3D-2mini.
Official Hugging Face model repository.
https://huggingface.co/tencent/Hunyuan3D-2mini.

[2] Tencent Hunyuan Team.
hunyuan3d-dit-v2-mini-turbo/config.yaml.
Official Hunyuan3D-2mini model configuration.
https://huggingface.co/tencent/Hunyuan3D-2mini/blob/main/hunyuan3d-dit-v2-mini-turbo/config.yaml.

References ii

[3] Tencent Hunyuan Team.
Hunyuan3D-2.
Official source-code repository.
https://github.com/Tencent-Hunyuan/Hunyuan3D-2.

[4] William E. Lorensen and Harvey E. Cline.
Marching Cubes: A High Resolution 3D Surface Construction Algorithm.
Computer Graphics, 21(4), 1987.